Docker Node.js App Containerization

1. Introduction

The purpose of this project is to understand how Docker can be used to containerize a Node.js application. Docker provides a lightweight, portable, and consistent environment where applications can run without worrying about underlying system differences.

In this project, we:

- Clone a Node.js application
- Create a Dockerfile
- Build a Docker image
- Run the container
- Access the application using the terminal (curl)
- Understand image layers, port mapping, and dependency installation

This project gives practical hands-on experience with Docker fundamentals used widely in DevOps and modern application deployment.

2. What is Docker?

Docker is a platform designed to help developers build, ship, and run applications inside isolated environments called **containers**. Containers include everything the application needs, such as code, libraries, dependencies, and runtime. This ensures the application behaves the same across all systems.

3. Technologies Used

Technology	Purpose
Node.js	Backend JavaScript runtime for the sample app
Docker	For building and running the application in containers
Linux OS (Ubuntu)	Execution environment for Docker commands
Yarn Package Manager	Used by the application to install dependencies

4. Project Objectives

The main objectives of the project are:

- To learn how to containerize a Node.js application
- To understand Dockerfile structure
- To build a custom Docker image
- To deploy and run an app using Docker containers
- To expose application ports using port mapping
- To run and manage containers efficiently

5. Step 1: Cloning the Node.js Application

To begin, we clone Docker's official sample application.

Command:

git clone https://github.com/docker/getting-started-app.git

Explanation:

- This downloads the repository to your Linux machine.
- A folder named **getting-started-app** is created.
- This folder contains the Node.js application (source code, package.json, configuration files).

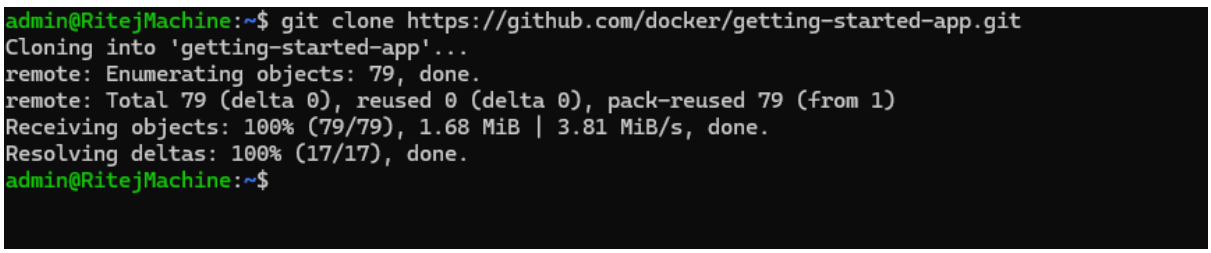

Fig 1- Cloning the repository

6. Step 2: Navigate into the Application Directory

Command:

cd getting-started-app

Explanation:

This places you inside the project folder where the application files exist.
You should see files like:

- package.json
- src/ folder
- README.md

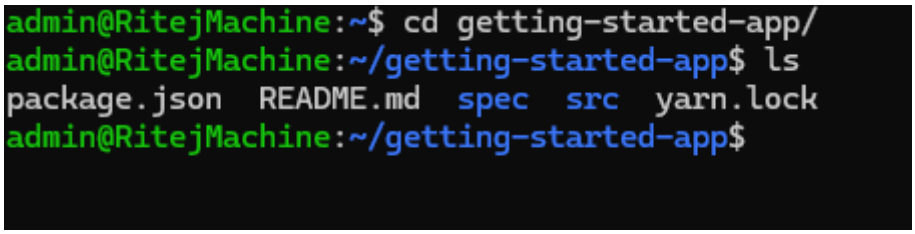

Fig 2- Project folder contents

7. Step 3: Creating the Dockerfile

A **Dockerfile** tells Docker how to build the container.

Command to create file:

vi Dockerfile

Paste the following:

```
# syntax=docker/dockerfile:1

FROM node:lts-alpine
WORKDIR /app
COPY . .
RUN yarn install --production
CMD ["node", "src/index.js"]
EXPOSE 3000
```

Explanation of Every Line

Line	Purpose
FROM node:lts-alpine	: Uses a lightweight Node.js base image
WORKDIR /app	: Creates and sets /app as working directory
COPY . .	: Copies local folder content into the container
RUN yarn install --production	: Installs required dependencies
CMD ["node", "src/index.js"]	: Starts Node.js application
EXPOSE 3000	: Informs Docker that the app runs on port 3000

This file is the most important part of containerization.

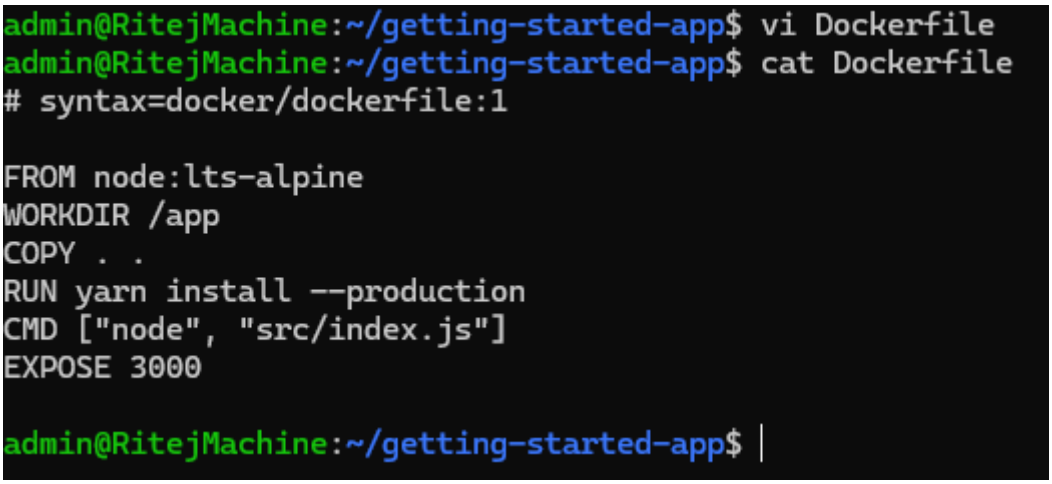

Fig 3- Dockerfile content

8. Step 4: Building the Docker Image

Command:

docker build -t getting-started .

Explanation:

- **docker build** → Creates an image from the Dockerfile
- **-t getting-started** → Gives the image a name (tag)
- **.** → Build context is the current directory

During the build process:

1. Docker downloads the base image (node:lts-alpine)

2. Creates layers for copying code
3. Installs dependencies using yarn
4. Prepares your application image

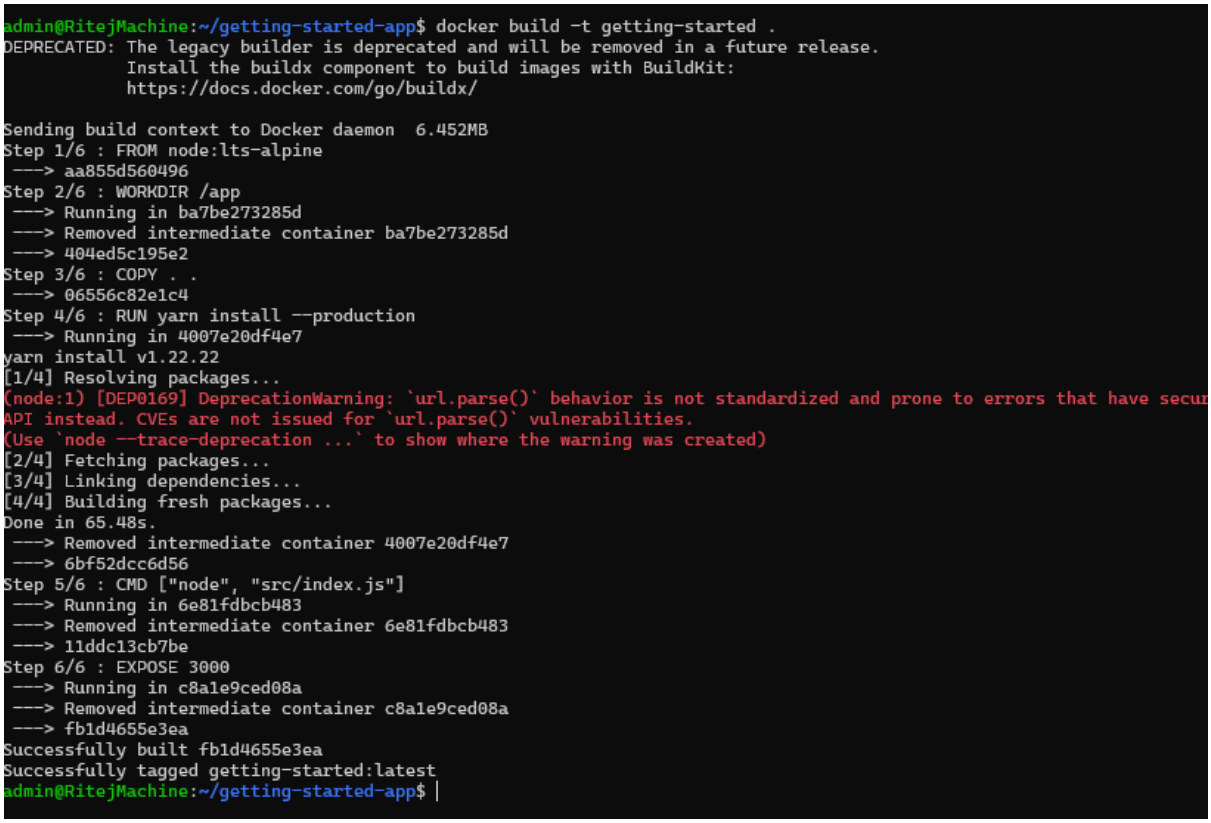

Fig 4- Build output

9. Step 5: Running the Container

Run the container using:

Command:

docker run -d -p 127.0.0.1:3000:3000 getting-started

Explanation of Flags:

Flag	Meaning
-d	Runs the container in detached mode (background)
-p 127.0.0.1:3000:3000	Maps your system port 3000 → container port 3000
getting-started	Image name to run

Now the app is running inside a container.

Fig 5- Container running

10. Step 6: Access the Application

You can check whether the application is running by using the curl command.

Command:

curl http://localhost:3000

If the container is running correctly, this command will return the HTML output of your Node.js application directly in the terminal.

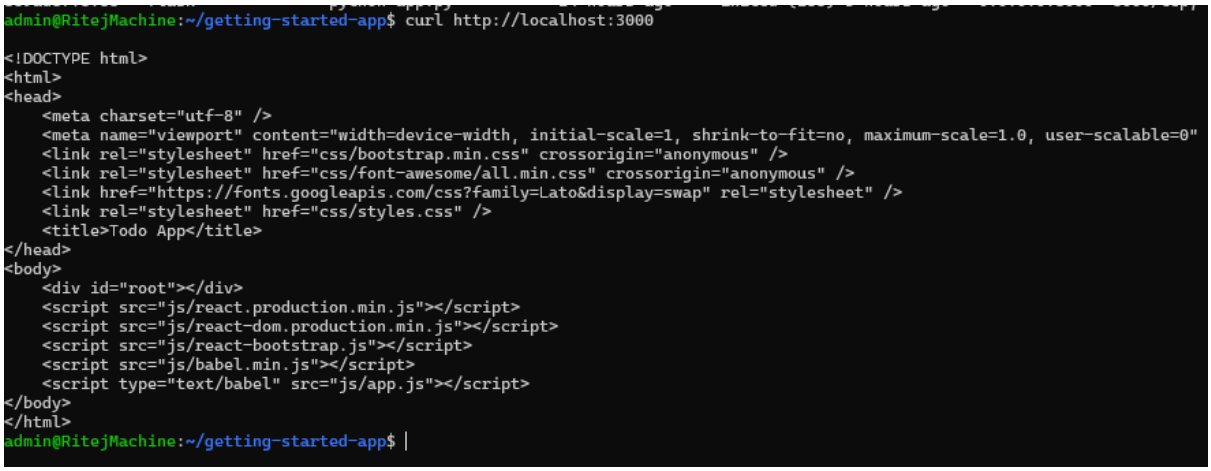

Fig 6- Web browser output

11. Step 7: Checking Running Containers

Command:

docker ps

Explanation:

This displays the list of currently running containers along with:

- Container ID
- Image name
- Ports
- Status

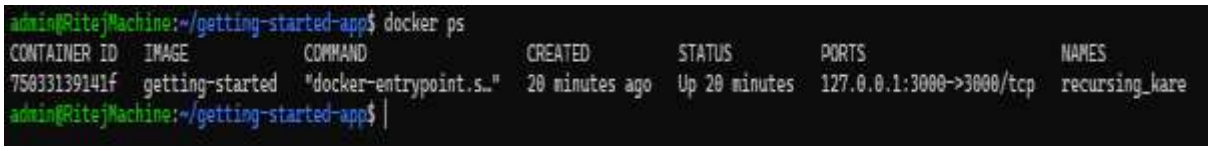

Fig 7- docker ps output

12. Understanding Docker Image Layers

A Docker image is built in multiple layers.
Each instruction in the Dockerfile creates a new layer.

Example:

- Base image layer
- Copy layer
- Dependencies layer
- Final runtime layer

Layers help in:

- Faster builds
- Smaller images
- Efficient caching

13. Benefits of Containerization

Benefit	Description
Consistency	Same behavior across systems
Isolation	App runs independent of host OS
Portability	Container runs anywhere (Linux, Windows, Cloud)
Scalability	Easy to scale up with more containers
Efficiency	Lightweight and fast

14. Conclusion

In this project, we successfully containerized a Node.js application using Docker. We learned:

- How to use Git to clone a repository
- How to create and write a Dockerfile
- How to build a custom Docker image
- How to run the application in a Docker container
- How to expose ports and access the application using the terminal
- How to manage containers and images

This project demonstrates practical Docker skills that are essential for DevOps, backend development, and deployment workflows. The steps completed here form the foundation for more advanced concepts such as Docker Compose, container orchestration, and cloud deployment.